Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

HWSW-Systeme

Übung 09: FIFOs und Tri-State-Modellierung

M.Sc. Eike Trumann, M.Sc. Jasper Homann,
Prof. Dr.-Ing. G. Payá Vayá

19.12.2025

1 Speicherkomponenten

2 First-In-First-Out (FIFO)

2.1 FIFO mit FSM beschreiben

Die Kontrollogik für eine FIFO mit einer Tiefe von 1024 Elementen soll in VHDL implementiert werden.

a) Wie viele Addressbits werden benötigt um den Speicher zu addressieren? Was ist die Gesamtgröße des Speichers in Byte, wenn jedes Element 32-bit groß ist?

b) Die Kontrollogik soll mithilfe von Pointern und einer FSM für den Über- bzw. Leerlaufschutz realisiert werden. Beschreiben Sie kurz die für die FSM benötigten Zustände.

c) Beschreiben Sie den FIFO Controller in VHDL.

```
library IEEE;
use IEEE.std_logic_1164.all;
use ieee.numeric_std.all;

entity fifo_sync is
port (
    clk, rst_n         : in  std_ulogic;         -- clock and reset
    wr_i, rd_i         : in  std_ulogic;         -- write- and read enable
    full_o, empty_o    : out std_ulogic;         -- status flags of the FIFO
    w_addr_o, r_addr_o : out std_ulogic_vector( downto 0) -- address to RAM Module
);
end entity;

architecture rtl of fifo_sync is

end architecture;
```

3 Tri-State-Modellierung

3.1 Multi-Valued Logic

a) Geben Sie alle 9 möglichen Signalwerte von `std_ulogic` bzw. `std_logic` an.

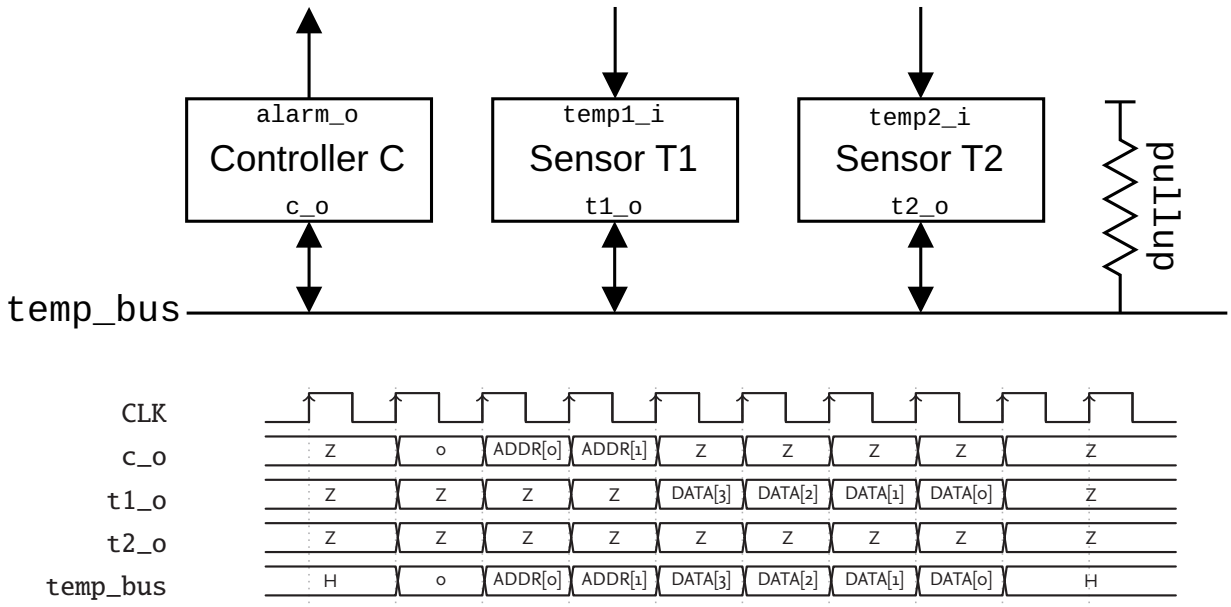
b) Das `std_logic` Signal `resolved_bus` hat zwei Treiber (`driver1` und `driver2`). Tragen Sie in der Tabelle den resultierenden Wert von `s` anhand der Resolution Function ein

driver1	driver2	resolved_bus
'Z'	'1'	
'Z'	'0'	
'Z'	'H'	
'Z'	'L'	
'1'	'H'	

driver1	driver2	resolved_bus
'1'	'0'	
'H'	'0'	
'L'	'H'	
'0'	'L'	
'Z'	'Z'	

3.2 Tri-State Bus

Zwei 4-bit Temperatursensoren (`T1` und `T2`) sollen über einen 1-bit Bus mit einem Controller (C) verbunden werden. Der Controller soll abwechselnd die Temperatur der beiden Sensoren lesen und das Signal `alarm_o` auf `'1'` setzen, wenn einer der beiden Sensoren einen Wert ≥ 8 ausgibt.



a) Implementieren Sie den kombinatorischen Prozess für die Controller FSM.

```
architecture rtl of temp_bus is
...
    -- Constants
    constant T1_ADDR : std_ulogic_vector(1 downto 0) := "01";
    constant T2_ADDR : std_ulogic_vector(1 downto 0) := "10";

    -- Controller Signals
    signal temp1_ff, temp1_nxt, temp2_ff, temp2_nxt, temp_recv_ff, temp_recv_nxt : std_ulogic_vector(3 do
    signal addr_ff, addr_nxt : std_ulogic_vector(1 downto 0);

    type controller_fsm_t is (RESET, START, ADDR1, ADDR2, RECV1, RECV2, RECV3, RECV4, STORE);
    signal controller_fsm_ff, controller_fsm_nxt : controller_fsm_t;

    signal c_o : std_logic;
...
begin
...
    controller_fsm: process (all) is
    begin

    end process;
...
end architecture;
```

b) Implementieren Sie den kombinatorischen Prozess für die Sensor FSM von T1.

```
architecture rtl of temp_bus is
...
    -- Sensor Signals
    type sensor_fsm_t is (IDLE, ADDR1, ADDR2, SEND1, SEND2, SEND3, SEND4);

    signal t1_temp_ff, t1_temp_nxt : std_ulogic_vector(3 downto 0);
    signal t1_active_ff, t1_active_nxt : std_ulogic;
    signal t1_fsm_ff, t1_fsm_nxt : sensor_fsm_t;
...
begin
...
    sensor1_fsm: process (all) is
    begin

    end process;
...
end architecture;
```